Skin-Disease Image Classification with MobileNetV2

Project notebook for training, evaluation, visualizations, and presentation

This notebook organizes the workflow into clear, numbered stages:

1. Confirm the local Python/TensorFlow environment and GPU visibility.
2. Extract and inspect the image dataset.
3. Explore class balance and example images.
4. Create a stratified 70% / 15% / 15% train-validation-test split.
5. Build the image pipeline.
6. Build a transfer-learning model with pretrained MobileNetV2.
7. Train the classification head, then fine-tune selected backbone layers.
8. Select a checkpoint using validation performance.
9. Evaluate once on the held-out test set and visualize the results.
10. Save the selected model and class-name mapping.

Important: Run the cells from top to bottom. Training may take a long time on a local runtime. Do not rerun the training cells just to regenerate plots if the existing `history_head`, `history_fine`, model, and test variables are still available.

This is an experimental image-classification project, **not a clinically validated diagnostic tool**.

Stage 1 — Check the local runtime and GPU

Run this stage first to confirm Colab is connected to your intended local Python environment.

```
# Stage 1A: Show the Python and TensorFlow versions used by this notebook.
import os
import sys
import platform
import time

import tensorflow as tf

print("Operating system:", platform.platform())
print("Python executable:", sys.executable)
print("Python version:", sys.version)
print("TensorFlow version:", tf.__version__)

# List devices TensorFlow can see. On this Windows setup, the DirectML
# plugin may expose compatible graphics adapters as GPU:0, GPU:1, etc.
GPU_DEVICES = tf.config.list_physical_devices("GPU")
print("\nTensorFlow GPU devices:")
for device in GPU_DEVICES:
    print(" ", device)
```

```
if not GPU_DEVICES:
    print("No GPU detected by TensorFlow. Training may use the CPU.")
```

```
Operating system: Windows-10-10.0.26300-SP0
Python executable: C:\tf-directml\Scripts\python.exe
Python version: 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
TensorFlow version: 2.10.0

TensorFlow GPU devices:
  PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')
  PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')
```

```
# Stage 1B: Run a small matrix-multiplication smoke test on each detected GPU.
# This confirms that a basic operation can execute on the selected device.
# It does not prove that every operation in the full model is GPU-supported.
for gpu_index in range(len(GPU_DEVICES)):
    device_name = f"/GPU:{gpu_index}"
    try:
        start_time = time.time()

        with tf.device(device_name):
            test_a = tf.random.normal((512, 512))
            test_b = tf.matmul(test_a, test_a)
            _ = test_b.numpy()  # Force the result to be computed.

        print(
            f"{device_name}: computation succeeded; "
            f"result device = {test_b.device}; "
            f"time = {time.time() - start_time:.2f}s"
        )
    except Exception as error:
        print(f"{device_name}: test failed: {error}")
```

```
/GPU:0: computation succeeded; result device = /job:localhost/replica:0/task:0/device:GPU:0; time = 0.42s
/GPU:1: computation succeeded; result device = /job:localhost/replica:0/task:0/device:GPU:1; time = 0.05s
```

Stage 2 — Imports and project configuration

Method: define file locations, image settings, class extensions, and plot-output folder. Change the ZIP path if your file is in a different location.

```
# Stage 2: Import libraries used for data handling, plotting, splitting,
# transfer learning, training callbacks, and evaluation.
import os
import shutil
import zipfile
import json
import random
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    precision_recall_fscore_support,
    accuracy_score
)

from tensorflow.keras import layers, Model
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ReduceLROnPlateau,
    ModelCheckpoint
)

# Update ZIP_PATH if skin_data.zip is saved in another folder.
ZIP_PATH = Path(r"C:\Users\Akhilesh\Downloads\skin_data.zip")

# The ZIP will be extracted into this local folder.
EXTRACT_TO = Path(r"C:\Users\Akhilesh\Datasets\skin_data")

# Model and data parameters.
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 16
SEED = 42
IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

# Save plots, checkpoints, and the final model next to the dataset folder.
OUTPUT_DIR = EXTRACT_TO.parent
FIGURE_DIR = OUTPUT_DIR / "skin_model_figures"
FIGURE_DIR.mkdir(parents=True, exist_ok=True)

# Reproducibility for Python and NumPy; shuffle/split seeds are set separately.
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("ZIP path:", ZIP_PATH)
print("Dataset extraction folder:", EXTRACT_TO)
print("Figure output folder:", FIGURE_DIR)
print("Image size:", IMAGE_SIZE, "| Batch size:", BATCH_SIZE)
```

```
ZIP path: C:\Users\Akhilesh\Downloads\skin_data.zip
Dataset extraction folder: C:\Users\Akhilesh\Datasets\skin_data
Figure output folder: C:\Users\Akhilesh\Datasets\skin_model_figures
Image size: (224, 224) | Batch size: 16
```

Stage 3 — Extract the dataset

Method: inspect the archive size and disk space before extraction. A marker file prevents accidental repeated extraction.

```
# Stage 3: Extract the ZIP file only if it has not already been extracted.
MARKER = EXTRACT_TO / ".extraction_complete"

if not ZIP_PATH.is_file():
    raise FileNotFoundError(
        f"ZIP file not found: {ZIP_PATH}\n"
        "Update ZIP_PATH to the actual location of skin_data.zip."
    )

if MARKER.exists():
    print("Dataset extraction marker found:", EXTRACT_TO)
else:
    # Inspect the ZIP before extracting it.
    with zipfile.ZipFile(ZIP_PATH, "r") as archive:
        entries = archive.infolist()
        uncompressed_size = sum(item.file_size for item in entries)
        file_count = sum(not item.is_dir() for item in entries)

    free_bytes = shutil.disk_usage(EXTRACT_TO.parent).free

    print(f"ZIP size: {ZIP_PATH.stat().st_size / 1024**3:.2f} GB")
    print(f"Estimated extracted size: {uncompressed_size / 1024**3:.2f} GB")
    print(f"Files inside ZIP: {file_count:,}")
    print(f"Free disk space: {free_bytes / 1024**3:.2f} GB")

    # Keep at least 1.5 GB of free space after extraction.
    reserve_bytes = int(1.5 * 1024**3)
    if free_bytes < uncompressed_size + reserve_bytes:
        raise RuntimeError(
            "Not enough free disk space. Free space or choose another drive."
        )

    # Do not delete an existing folder automatically.
    # If an incomplete extraction exists, inspect it before cleaning it manually.
    EXTRACT_TO.mkdir(parents=True, exist_ok=True)

    print("Extracting dataset...")
    start_time = time.time()
    with zipfile.ZipFile(ZIP_PATH, "r") as archive:
        archive.extractall(EXTRACT_TO)

    MARKER.write_text("Extraction completed successfully.", encoding="utf-8")
    print(f"Extraction completed in {(time.time() - start_time) / 60:.2f} minutes.")

print("Top-level contents:", [item.name for item in EXTRACT_TO.iterdir()][:20])
```

```
Dataset extraction marker found: C:\Users\Akhilesh\Datasets\skin_data
Top-level contents: ['.extraction_complete', 'skin data']
```

Stage 4 — Locate the disease-class folders

Method: automatically find a folder whose immediate subfolders contain image files. Confirm the printed folder and class list look correct before continuing.

```
    # Stage 4: Identify the folder that contains one subfolder per disease class.
    # Use one directory traversal and aggregate image counts upward for efficiency.
    def find_class_root(base_path):
        """Find a directory with at least two immediate child folders containing images."""
        base_path = Path(base_path)

        all_dirs = []
        child_dirs = {}
        image_counts = {}

        for root, dirs, files in os.walk(base_path):
            current = Path(root)
            all_dirs.append(current)
            child_dirs[current] = [current / name for name in dirs]

            # Count non-empty image files located directly in this folder.
            image_counts[current] = sum(
                1 for name in files
                if Path(name).suffix.lower() in IMAGE_EXTENSIONS
                and (current / name).stat().st_size > 0
            )

        # Aggregate each folder's image total into its parent, deepest folders first.
        for current in sorted(all_dirs, key=lambda path: len(path.parts), reverse=True):
            if current != base_path:
                image_counts[current.parent] = (
                    image_counts.get(current.parent, 0) + image_counts[current]
                )

        candidates = []
        for current in all_dirs:
            # A child folder counts as a class candidate if it contains image files
            # anywhere below it.
            class_count = sum(
                image_counts.get(child, 0) > 0
                for child in child_dirs[current]
            )
            if class_count >= 2:
                candidates.append((class_count, len(current.parts), current))

        if not candidates:
            raise ValueError(
                "Could not find a folder containing at least two "
                "disease-class subfolders with images."
            )

        # Prefer the directory with most image-containing child folders.
        candidates.sort(key=lambda item: (item[0], item[1]), reverse=True)
        return candidates[0][2]


    def count_images(folder):
        """Count non-empty image files recursively inside a folder."""
        return sum(
            1 for file in folder.rglob("*")
```

```
        if file.is_file()
        and file.suffix.lower() in IMAGE_EXTENSIONS
        and file.stat().st_size > 0
    )


dataset_path = find_class_root(EXTRACT_TO)

class_names = sorted([
    folder.name
    for folder in dataset_path.iterdir()
    if folder.is_dir() and count_images(folder) > 0
])

print("Detected class root:", dataset_path)
print("Number of classes:", len(class_names))
for index, class_name in enumerate(class_names):
    print(f"{index}: {class_name} ({count_images(dataset_path / class_name):,} images)")

if len(class_names) < 2:
    raise ValueError("At least two class folders are required.")
```

```
Detected class root: C:\Users\Akhilesh\Datasets\skin_data\skin data
Number of classes: 10
0: 1. Eczema 1677 (1,677 images)
1: 10. Warts Molluscum and other Viral Infections - 2103 (2,103 images)
2: 2. Melanoma 15.75k (3,140 images)
3: 3. Atopic Dermatitis - 1.25k (1,257 images)
4: 4. Basal Cell Carcinoma (BCC) 3323 (3,323 images)
5: 5. Melanocytic Nevi (NV) - 7970 (7,970 images)
6: 6. Benign Keratosis-like Lesions (BKL) 2624 (2,079 images)
7: 7. Psoriasis pictures Lichen Planus and related diseases - 2k (2,055 images)
8: 8. Seborrheic Keratoses and other Benign Tumors - 1.8k (1,847 images)
9: 9. Tinea Ringworm Candidiasis and other Fungal Infections - 1.7k (1,702 images)
```

Stage 5 — Build the image inventory and inspect class balance

Graph 1: images per class. Run this before splitting or training. It helps explain class imbalance to the judges.

```
# Stage 5: Create the list of valid image paths and their integer class labels.
# Empty image files are skipped and removed from the local extracted copy.
file_paths = []
labels = []
empty_files = []

for class_index, class_name in enumerate(class_names):
    class_folder = dataset_path / class_name

    for file in sorted(class_folder.rglob("*")):
        if file.is_file() and file.suffix.lower() in IMAGE_EXTENSIONS:
            if file.stat().st_size == 0:
                empty_files.append(file)
            else:
                file_paths.append(str(file))
                labels.append(class_index)
```

```
for file in empty_files:
    file.unlink()

print("Empty files removed:", len(empty_files))
print("Total usable images:", f"{len(file_paths):,}")

# Build a table of images in each class.
class_counts = (
    pd.Series([class_names[label] for label in labels])
    .value_counts()
    .reindex(class_names, fill_value=0)
)

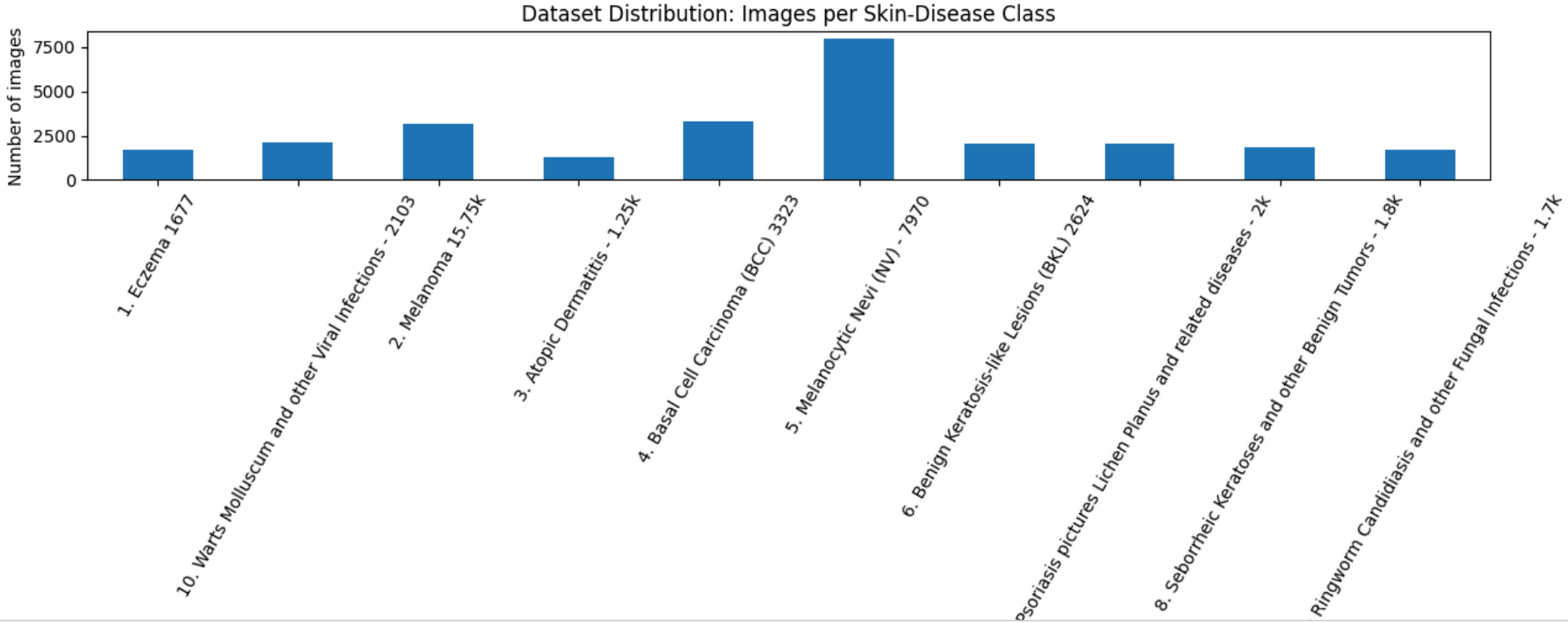
display(class_counts.rename("image_count").to_frame())

# Graph 1: dataset class distribution.
fig, ax = plt.subplots(figsize=(13, 6))
class_counts.plot(kind="bar", ax=ax)
ax.set_title("Dataset Distribution: Images per Skin-Disease Class")
ax.set_xlabel("Disease class")
ax.set_ylabel("Number of images")
ax.tick_params(axis="x", labelrotation=60)
plt.tight_layout()
plt.savefig(FIGURE_DIR / "01_class_distribution.png", dpi=200, bbox_inches="tight")
plt.show()

if not file_paths:
    raise ValueError("No valid image files were found.")
```

```
Empty files removed: 0
Total usable images: 27,153
```

	image_count
1. Eczema 1677	1677
10. Warts Molluscum and other Viral Infections - 2103	2103
2. Melanoma 15.75k	3140
3. Atopic Dermatitis - 1.25k	1257
4. Basal Cell Carcinoma (BCC) 3323	3323
5. Melanocytic Nevi (NV) - 7970	7970
6. Benign Keratosis-like Lesions (BKL) 2624	2079
7. Psoriasis pictures Lichen Planus and related diseases - 2k	2055
8. Seborrheic Keratoses and other Benign Tumors - 1.8k	1847
9. Tinea Ringworm Candidiasis and other Fungal Infections - 1.7k	1702



Stage 6 — Show representative images from each class

Graph 2: sample image grid. Use this to show judges what the input data looks like before preprocessing.

```
# Stage 6: Display one example image from each class.
# This is a visual quality check, not model evaluation.
```

```
sample_paths = []
sample_labels = []

for class_index, class_name in enumerate(class_names):
    class_files = [
        path for path, label in zip(file_paths, labels)
        if label == class_index
    ]
    if class_files:
        sample_paths.append(class_files[0])
        sample_labels.append(class_index)

n_images = len(sample_paths)
n_cols = 3
n_rows = int(np.ceil(n_images / n_cols))

fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 4 * n_rows))
axes = np.array(axes).reshape(-1)

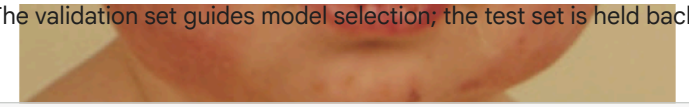
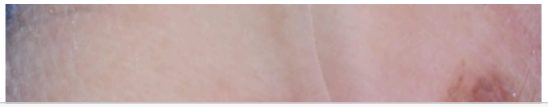
for ax, path, label in zip(axes, sample_paths, sample_labels):
    try:
        with Image.open(path) as img:
            ax.imshow(img.convert("RGB"))
        ax.set_title(class_names[label], fontsize=9)
    except Exception as error:
        ax.set_title(f"Could not load image\n{error}", fontsize=8)
    ax.axis("off")

# Hide unused axes if class count is not a multiple of n_cols.
for ax in axes[len(sample_paths):]:
    ax.axis("off")

fig.suptitle("Representative Dataset Images by Class", fontsize=16)
plt.tight_layout()
plt.savefig(FIGURE_DIR / "02_sample_images_by_class.png", dpi=200, bbox_inches="tight")
plt.show()
```

Stage 7 — Create a stratified train/validation/test split

Method: 70% training, 15% validation, 15% test, stratified by class. The validation set guides model selection; the test set is held back for final assessment.

```
# Stage 7: Split the file paths and labels without loading every image into RAM.
X_train, X_temp, y_train, y_temp = train_test_split(
    file_paths,
    labels,
    test_size=0.30,
    random_state=SEED,
    stratify=labels
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=SEED,
    stratify=y_temp
)

split_counts = pd.Series({
    "Training": len(X_train),
    "Validation": len(X_val),
    "Testing": len(X_test)
})

print(split_counts.to_string())
print("Total:", sum(split_counts))

# Graph 3: show the overall split sizes.
fig, ax = plt.subplots(figsize=(8, 5))
split_counts.plot(kind="bar", ax=ax)
ax.set_title("Dataset Split Sizes")
ax.set_xlabel("Subset")
ax.set_ylabel("Number of images")
ax.tick_params(axis="x", labelrotation=0)
for patch in ax.patches:
    ax.annotate(
        f"{int(patch.get_height()):,}",
        (patch.get_x() + patch.get_width() / 2, patch.get_height()),
        ha="center", va="bottom"
    )
plt.tight_layout()
```

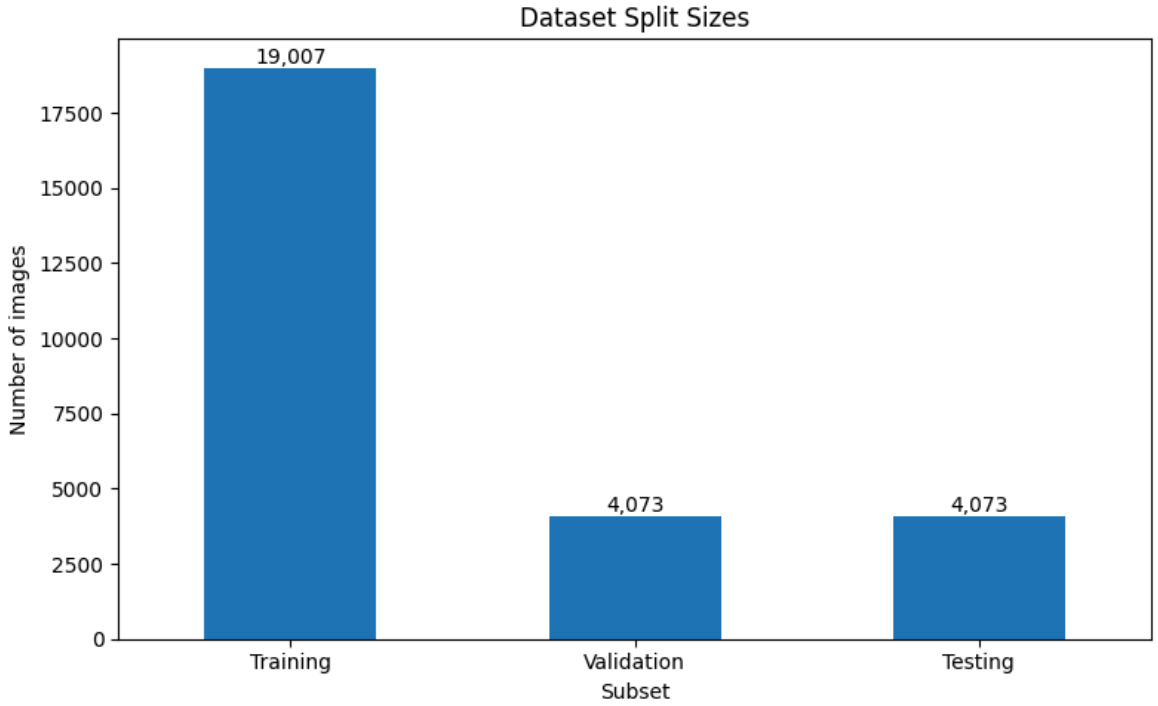
```
plt.savefig(FIGURE_DIR / "03_data_split_sizes.png", dpi=200, bbox_inches="tight")
plt.show()

# Optional extra graph: verify that every subset retains a similar class mix.
split_class_counts = pd.DataFrame({
    "Training": pd.Series([class_names[i] for i in y_train]).value_counts(),
    "Validation": pd.Series([class_names[i] for i in y_val]).value_counts(),
    "Testing": pd.Series([class_names[i] for i in y_test]).value_counts()
}).reindex(class_names, fill_value=0)

fig, ax = plt.subplots(figsize=(14, 6))
split_class_counts.plot(kind="bar", ax=ax)
ax.set_title("Per-Class Image Counts in Each Data Split")
ax.set_xlabel("Disease class")
ax.set_ylabel("Number of images")
ax.tick_params(axis="x", labelrotation=60)
plt.tight_layout()
plt.savefig(FIGURE_DIR / "04_class_counts_by_split.png", dpi=200, bbox_inches="tight")
plt.show()
```



```
Testing         4073
Total: 27153
```





Stage 8 — Build the TensorFlow input pipeline

Method: read one image at a time, decode as RGB, resize to 224×224, batch the images, and prefetch. Pixels remain in the 0–255 range here because MobileNetV2 preprocessing is included inside the model.

```
# Stage 8: Create efficient dataset-loading functions.
AUTOTUNE = tf.data.AUTOTUNE
NUM_CLASSES = len(class_names)

def load_image(path, label):
    """Read and resize an image; return float pixels and integer label."""
    image_bytes = tf.io.read_file(path)
    image = tf.io.decode_image(
        image_bytes,
        channels=3,
        expand_animations=False
    )
    image.set_shape([None, None, 3])
    image = tf.image.resize(image, IMAGE_SIZE)
    image = tf.cast(image, tf.float32)  # Keep 0–255 for MobileNetV2 preprocessing.
    return image, tf.cast(label, tf.int32)

def make_dataset(paths, labels, training=False):
    """Create a batched, prefetched tf.data dataset."""
    dataset = tf.data.Dataset.from_tensor_slices((paths, labels))

    if training:
        dataset = dataset.shuffle(
            buffer_size=min(len(paths), 10000),
            seed=SEED,
            reshuffle_each_iteration=True
        )

    dataset = dataset.map(load_image, num_parallel_calls=AUTOTUNE)
    dataset = dataset.batch(BATCH_SIZE)
    dataset = dataset.prefetch(AUTOTUNE)
    return dataset

train_ds = make_dataset(X_train, y_train, training=True)
val_ds = make_dataset(X_val, y_val)
test_ds = make_dataset(X_test, y_test)

print("Train batches:", tf.data.experimental.cardinality(train_ds).numpy())
print("Validation batches:", tf.data.experimental.cardinality(val_ds).numpy())
print("Test batches:", tf.data.experimental.cardinality(test_ds).numpy())
```

```
Train batches: 1188
Validation batches: 255
Test batches: 255
```

Stage 9 — Visualize preprocessed input images

Graph 5: sample preprocessed batch. Place this before model creation to demonstrate what the model receives as input.

```
# Stage 9: Display a batch after resizing. Divide by 255 only for visualization.
for batch_images, batch_labels in train_ds.take(1):
    n_show = min(12, int(batch_images.shape[0]))
    cols = 4
    rows = int(np.ceil(n_show / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(12, 3 * rows))
    axes = np.array(axes).reshape(-1)

    for index in range(n_show):
        axes[index].imshow(
            np.clip(batch_images[index].numpy() / 255.0, 0, 1)
        )
        label_index = int(batch_labels[index].numpy())
        axes[index].set_title(class_names[label_index], fontsize=8)
        axes[index].axis("off")

    for ax in axes[n_show:]:
        ax.axis("off")

    fig.suptitle("Resized Training Images (224 × 224)", fontsize=14)
    plt.tight_layout()
    plt.savefig(FIGURE_DIR / "05_preprocessed_batch.png", dpi=200, bbox_inches="tight")
    plt.show()
```



Stage 10 — Build the transfer-learning model

Method: use MobileNetV2 pretrained on ImageNet as the feature extractor. Initially freeze its weights; add data augmentation, global average pooling, dropout, and a softmax classifier for the skin-disease classes.

```
# Stage 10: Create the MobileNetV2 transfer-learning model.
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.10)
```

```
], name="data_augmentation")

# Load ImageNet-pretrained MobileNetV2 without its original 1000-class head.
base_model = MobileNetV2(
    input_shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3),
    include_top=False,
    weights="imagenet"
)
base_model.trainable = False  # Phase 1: freeze pretrained backbone.

inputs = layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3), name="image_input")
x = data_augmentation(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D(name="global_average_pooling")(x)
x = layers.Dropout(0.35, name="dropout")(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax", name="disease_probabilities")(x)

model = Model(inputs, outputs, name="skin_disease_mobilenetv2")

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

```
Model: "skin_disease_mobilenetv2"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 image_input (InputLayer)    [(None, 224, 224, 3)]     0

 data_augmentation (Sequenti  (None, 224, 224, 3)      0
 al)

 tf.math.truediv (TFOpLambda  (None, 224, 224, 3)      0
 )

 tf.math.subtract (TFOpLambd  (None, 224, 224, 3)      0
 a)

 mobilenetv2_1.00_224 (Funct  (None, 7, 7, 1280)       2257984
 ional)

 global_average_pooling (Glo  (None, 1280)             0
 balAveragePooling2D)

 dropout (Dropout)           (None, 1280)              0

 disease_probabilities (Dens  (None, 10)               12810
 e)

=================================================================
Total params: 2,270,794
Trainable params: 12,810
Non-trainable params: 2,257,984
_________________________________________________________________
```

Stage 11 — Plot a conceptual model architecture

Graph 6: model pipeline diagram. This is a presentation-friendly schematic of the principal stages, not a drawing of every internal MobileNetV2 convolution.

```
# Stage 11: Draw a conceptual architecture diagram using Matplotlib.
# This does not require Graphviz or pydot.
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

architecture_steps = [
    ("Input image", "224 × 224 × 3"),
    ("Data augmentation", "Flip / rotate / zoom"),
    ("MobileNetV2", "ImageNet pretrained"),
    ("Global average pooling", "Compress feature maps"),
    ("Dropout", "Rate = 0.35"),
    ("Softmax classifier", f"{NUM_CLASSES} classes")
]

fig, ax = plt.subplots(figsize=(16, 4))
ax.set_xlim(0, len(architecture_steps) * 2.4)
ax.set_ylim(0, 3)
ax.axis("off")

box_width = 1.9
box_height = 1.25
gap = 0.45

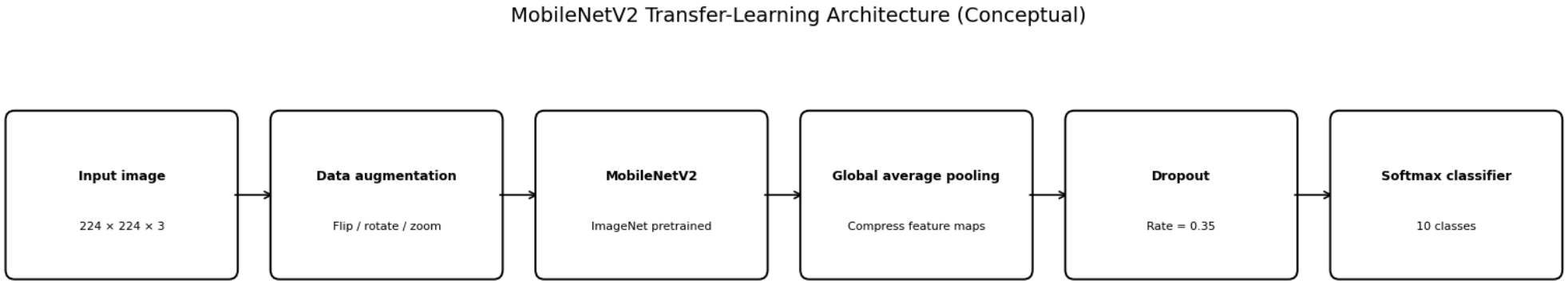
for i, (title, subtitle) in enumerate(architecture_steps):
    x = 0.25 + i * (box_width + gap)
    y = 1.0

    box = FancyBboxPatch(
        (x, y), box_width, box_height,
        boxstyle="round,pad=0.08",
        linewidth=1.4, edgecolor="black", facecolor="white"
    )
    ax.add_patch(box)
    ax.text(x + box_width / 2, y + 0.78, title,
            ha="center", va="center", fontsize=9, fontweight="bold", wrap=True)
    ax.text(x + box_width / 2, y + 0.36, subtitle,
            ha="center", va="center", fontsize=8, wrap=True)

    if i < len(architecture_steps) - 1:
        arrow = FancyArrowPatch(
            (x + box_width + 0.03, y + box_height / 2),
            (x + box_width + gap - 0.03, y + box_height / 2),
            arrowstyle="->", mutation_scale=12, linewidth=1.2
        )
        ax.add_patch(arrow)

ax.set_title("MobileNetV2 Transfer-Learning Architecture (Conceptual)", fontsize=14)
plt.tight_layout()
```

```
plt.savefig(FIGURE_DIR / "06_model_architecture.png", dpi=200, bbox_inches="tight")
plt.show()
```



Stage 12 — Calculate class weights

Method: balanced class weights increase the training loss contribution from less frequent classes. This can help imbalance, but does not guarantee that every class will improve.

```
# Stage 12: Compute weights from the training labels only.
unique_classes = np.unique(np.asarray(y_train))

weights = compute_class_weight(
    class_weight="balanced",
    classes=unique_classes,
    y=np.asarray(y_train)
)

class_weight_dict = {
    int(class_id): float(weight)
    for class_id, weight in zip(unique_classes, weights)
}

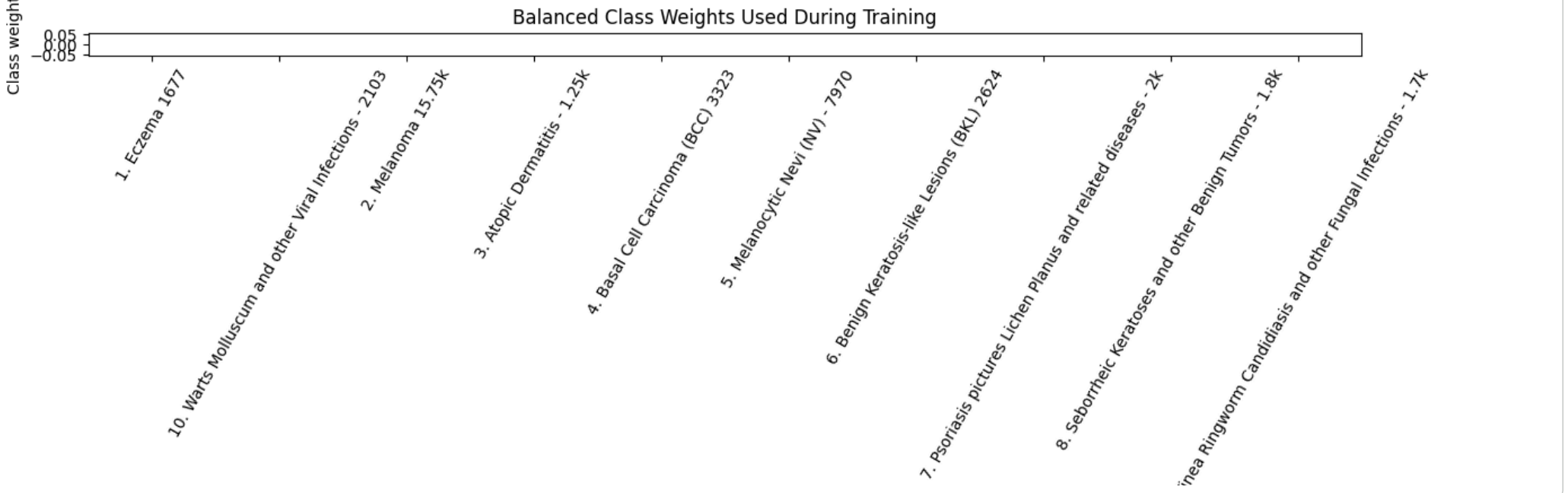
print("Class weights (integer label -> weight):")
print(class_weight_dict)

# Graph 7: visualize the weights used during training.
weight_series = pd.Series(
    class_weight_dict,
    index=[class_names[i] for i in class_weight_dict.keys()]
)

fig, ax = plt.subplots(figsize=(13, 5))
weight_series.plot(kind="bar", ax=ax)
ax.set_title("Balanced Class Weights Used During Training")
ax.set_xlabel("Disease class")
ax.set_ylabel("Class weight")
ax.tick_params(axis="x", labelrotation=60)
plt.tight_layout()
```

```
plt.savefig(FIGURE_DIR / "07_class_weights.png", dpi=200, bbox_inches="tight")
plt.show()
```

```
Class weights (integer label -> weight):
{0: 1.6189948892674617, 1: 1.2912364130434784, 2: 0.8647406733393994, 3: 2.1598863636363634, 4: 0.8171539122957867, 5: 0.3406882953934397, 6: 1.3063230240549828, 7: 1.32084781097984
```



Stage 13 — Phase 1 training: train the new classification head

Method: keep MobileNetV2 frozen and train the new classification layers. Callbacks stop training when validation loss no longer improves, reduce the learning rate when progress plateaus, and save the best validation-loss checkpoint.

Runtime note: this cell can take many minutes. Run it once; do not rerun to generate plots.

```
# Stage 13: Prepare output paths and reusable callbacks.
SAVE_DIR = OUTPUT_DIR
HEAD_PATH = SAVE_DIR / "best_skin_mobilenetv2_head.keras"
FINE_PATH = SAVE_DIR / "best_skin_mobilenetv2_finetuned.keras"

def get_callbacks(checkpoint_path, patience=4):
    """Return callbacks for best-checkpoint saving and training control."""
    return [
        EarlyStopping(
            monitor="val_loss",
            patience=patience,
            restore_best_weights=True,
            verbose=1
        ),
        ReduceLROnPlateau(
            monitor="val_loss",
            factor=0.5,
            patience=2,
```

```
            min_lr=1e-7,
            verbose=1
        ),
        ModelCheckpoint(
            str(checkpoint_path),
            monitor="val_loss",
            save_best_only=True,
            mode="min",
            verbose=1
        )
    ]

# Phase 1: train the new classifier while keeping the pretrained backbone frozen.
history_head = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=12,
    class_weight=class_weight_dict,
    callbacks=get_callbacks(HEAD_PATH, patience=4),
    verbose=1
)

print("Best head checkpoint:", HEAD_PATH)
print("Checkpoint exists:", HEAD_PATH.exists())
```

```
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
1188/1188 [==============================] - ETA: 0s - loss: 1.5281 - accuracy: 0.5032
Epoch 1: val_loss improved from inf to 1.01686, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_head.keras
1188/1188 [==============================] - 619s 512ms/step - loss: 1.5281 - accuracy: 0.5032 - val_loss: 1.0169 - val_accuracy: 0.6096 - lr: 0.0010
Epoch 2/12
1188/1188 [==============================] - ETA: 0s - loss: 1.2820 - accuracy: 0.5722
Epoch 2: val_loss improved from 1.01686 to 0.97047, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_head.keras
1188/1188 [==============================] - 624s 525ms/step - loss: 1.2820 - accuracy: 0.5722 - val_loss: 0.9705 - val_accuracy: 0.6273 - lr: 0.0010
Epoch 3/12
1188/1188 [==============================] - ETA: 0s - loss: 1.2436 - accuracy: 0.5877
Epoch 3: val_loss did not improve from 0.97047
1188/1188 [==============================] - 488s 410ms/step - loss: 1.2436 - accuracy: 0.5877 - val_loss: 0.9825 - val_accuracy: 0.6278 - lr: 0.0010
Epoch 4/12
1188/1188 [==============================] - ETA: 0s - loss: 1.2161 - accuracy: 0.6010
```

```
Epoch 9/12
1188/1188 [==============================] - ETA: 0s - loss: 1.1119 - accuracy: 0.6254
Epoch 9: val_loss did not improve from 0.88776
1188/1188 [==============================] - 277s 233ms/step - loss: 1.1119 - accuracy: 0.6254 - val_loss: 0.9001 - val_accuracy: 0.6573 - lr: 5.0000e-04
Epoch 10/12
1188/1188 [==============================] - ETA: 0s - loss: 1.1141 - accuracy: 0.6267
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 10: val_loss did not improve from 0.88776
1188/1188 [==============================] - 293s 246ms/step - loss: 1.1141 - accuracy: 0.6267 - val_loss: 0.9177 - val_accuracy: 0.6457 - lr: 5.0000e-04
Epoch 11/12
1188/1188 [==============================] - ETA: 0s - loss: 1.0729 - accuracy: 0.6400
Epoch 11: val_loss did not improve from 0.88776
1188/1188 [==============================] - 291s 245ms/step - loss: 1.0729 - accuracy: 0.6400 - val_loss: 0.8976 - val_accuracy: 0.6521 - lr: 2.5000e-04
Epoch 12/12
1188/1188 [==============================] - ETA: 0s - loss: 1.0671 - accuracy: 0.6414Restoring model weights from the end of the best epoch: 8.

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

Epoch 12: val_loss did not improve from 0.88776
1188/1188 [==============================] - 292s 246ms/step - loss: 1.0671 - accuracy: 0.6414 - val_loss: 0.9254 - val_accuracy: 0.6428 - lr: 2.5000e-04
Epoch 12: early stopping
Best head checkpoint: C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_head.keras
Checkpoint exists: True
```

Stage 14 — Graph the Phase 1 learning curves

Graph : training and validation accuracy/loss. Run immediately after Stage 13. These graphs help identify improvement, plateaus, and possible overfitting.

```
# Stage 14: Visualize training history for the frozen-backbone phase.
def plot_history(history, title_prefix, file_prefix):
    """Plot training/validation accuracy and loss from a Keras History object."""
    history_data = history.history
    epochs = range(1, len(history_data["loss"]) + 1)

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Accuracy plot.
    axes[0].plot(epochs, history_data["accuracy"], marker="o", label="Training accuracy")
    axes[0].plot(epochs, history_data["val_accuracy"], marker="o", label="Validation accuracy")
    axes[0].set_title(f"{title_prefix}: Accuracy")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Accuracy")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

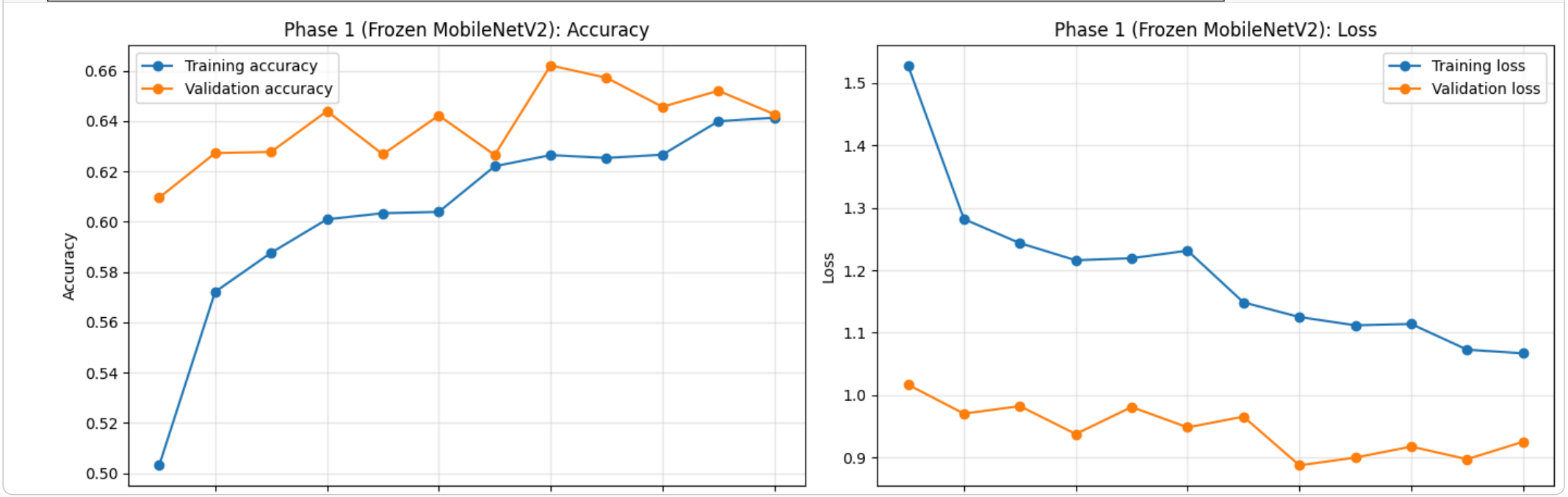
    # Loss plot.
    axes[1].plot(epochs, history_data["loss"], marker="o", label="Training loss")
    axes[1].plot(epochs, history_data["val_loss"], marker="o", label="Validation loss")
    axes[1].set_title(f"{title_prefix}: Loss")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Loss")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
```

```
    plt.savefig(FIGURE_DIR / f"{file_prefix}_learning_curves.png", dpi=200, bbox_inches="tight")
    plt.show()

    best_epoch = int(np.argmin(history_data["val_loss"]) + 1)
    print(f"Best validation-loss epoch: {best_epoch}")
    print(f"Best validation loss: {min(history_data['val_loss']):.4f}")
    print(f"Best validation accuracy: {max(history_data['val_accuracy']) * 100:.2f}%")

plot_history(history_head, "Phase 1 (Frozen MobileNetV2)", "08_phase1")
```



Stage 15 — Phase 2 fine-tuning: unfreeze only the final backbone layers

Method: load the saved Phase 1 checkpoint; unfreeze the final 30 MobileNetV2 layers; keep batch-normalization layers frozen; and train at a much smaller learning rate. Fine-tuning may improve the model, but improvement is not guaranteed.

```
# Stage 15: Load the best Phase 1 checkpoint before fine-tuning.
if not HEAD_PATH.exists():
    raise FileNotFoundError(
        f"Checkpoint missing: {HEAD_PATH}. Complete Stage 13 first."
    )

model = tf.keras.models.load_model(str(HEAD_PATH))

# Find the nested MobileNetV2 backbone in the Functional model.
base_model = next(
    (
        layer for layer in model.layers
        if isinstance(layer, tf.keras.Model)
        and "mobilenetv2" in layer.name.lower()
```

```
    ),
    None
)

if base_model is None:
    raise ValueError(
        "MobileNetV2 backbone was not found. Run model.summary() "
        "and verify that this is the MobileNetV2 model."
    )

# Fine-tuning: allow the final 30 layers to adapt to this dataset.
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False

# Keep BatchNormalization statistics stable during fine-tuning.
for layer in base_model.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False

# Recompile after changing trainable layers.
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=8,
    class_weight=class_weight_dict,
    callbacks=get_callbacks(FINE_PATH, patience=3),
    verbose=1
)

print("Best fine-tuned checkpoint:", FINE_PATH)
print("Checkpoint exists:", FINE_PATH.exists())
```

```
Epoch 1/8
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting RngReadAndSkip cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting Bitcast cause there is no registered converter for this op.
```

```
WARNING:tensorflow:Using a while_loop for converting StatelessRandomUniformV2 cause there is no registered converter for this op.
WARNING:tensorflow:Using a while_loop for converting ImageProjectiveTransformV3 cause there is no registered converter for this op.
1188/1188 [==============================] - ETA: 0s - loss: 1.0761 - accuracy: 0.6378
Epoch 1: val_loss improved from inf to 0.87729, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 310s 254ms/step - loss: 1.0761 - accuracy: 0.6378 - val_loss: 0.8773 - val_accuracy: 0.6639 - lr: 1.0000e-05
Epoch 2/8
1188/1188 [==============================] - ETA: 0s - loss: 1.0015 - accuracy: 0.6658
Epoch 2: val_loss improved from 0.87729 to 0.83654, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 506s 426ms/step - loss: 1.0015 - accuracy: 0.6658 - val_loss: 0.8365 - val_accuracy: 0.6771 - lr: 1.0000e-05
Epoch 3/8
1188/1188 [==============================] - ETA: 0s - loss: 0.9451 - accuracy: 0.6872
Epoch 3: val_loss improved from 0.83654 to 0.80025, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 806s 679ms/step - loss: 0.9451 - accuracy: 0.6872 - val_loss: 0.8003 - val_accuracy: 0.6946 - lr: 1.0000e-05
Epoch 4/8
1188/1188 [==============================] - ETA: 0s - loss: 0.8953 - accuracy: 0.7019
Epoch 4: val_loss did not improve from 0.80025
1188/1188 [==============================] - 755s 636ms/step - loss: 0.8953 - accuracy: 0.7019 - val_loss: 0.8144 - val_accuracy: 0.6892 - lr: 1.0000e-05
Epoch 5/8
1188/1188 [==============================] - ETA: 0s - loss: 0.8546 - accuracy: 0.7193
Epoch 5: val_loss improved from 0.80025 to 0.79441, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 459s 387ms/step - loss: 0.8546 - accuracy: 0.7193 - val_loss: 0.7944 - val_accuracy: 0.6943 - lr: 1.0000e-05
Epoch 6/8
1188/1188 [==============================] - ETA: 0s - loss: 0.8137 - accuracy: 0.7332
Epoch 6: val_loss improved from 0.79441 to 0.79359, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 566s 476ms/step - loss: 0.8137 - accuracy: 0.7332 - val_loss: 0.7936 - val_accuracy: 0.7029 - lr: 1.0000e-05
Epoch 7/8
1188/1188 [==============================] - ETA: 0s - loss: 0.7823 - accuracy: 0.7456
Epoch 7: val_loss improved from 0.79359 to 0.78377, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 305s 257ms/step - loss: 0.7823 - accuracy: 0.7456 - val_loss: 0.7838 - val_accuracy: 0.7027 - lr: 1.0000e-05
Epoch 8/8
1188/1188 [==============================] - ETA: 0s - loss: 0.7431 - accuracy: 0.7618
Epoch 8: val_loss improved from 0.78377 to 0.75561, saving model to C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
1188/1188 [==============================] - 422s 355ms/step - loss: 0.7431 - accuracy: 0.7618 - val_loss: 0.7556 - val_accuracy: 0.7123 - lr: 1.0000e-05
Best fine-tuned checkpoint: C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
Checkpoint exists: True
```

Stage 16 — Graph the fine-tuning learning curves

Graph 9: fine-tuning accuracy and loss. Run after Stage 15. Compare this with the Phase 1 graphs; choose the checkpoint based on validation results, not training accuracy alone.

```
# Stage 16: Plot the history for the fine-tuning phase.
plot_history(history_fine, "Phase 2 (Fine-tuned MobileNetV2)", "09_phase2")

# Optional combined plot with a vertical divider between the two phases.
# Epoch numbers in Phase 2 continue after Phase 1 for presentation only.
head_epochs = len(history_head.history["loss"])
fine_epochs = len(history_fine.history["loss"])
all_epochs = list(range(1, head_epochs + fine_epochs + 1))

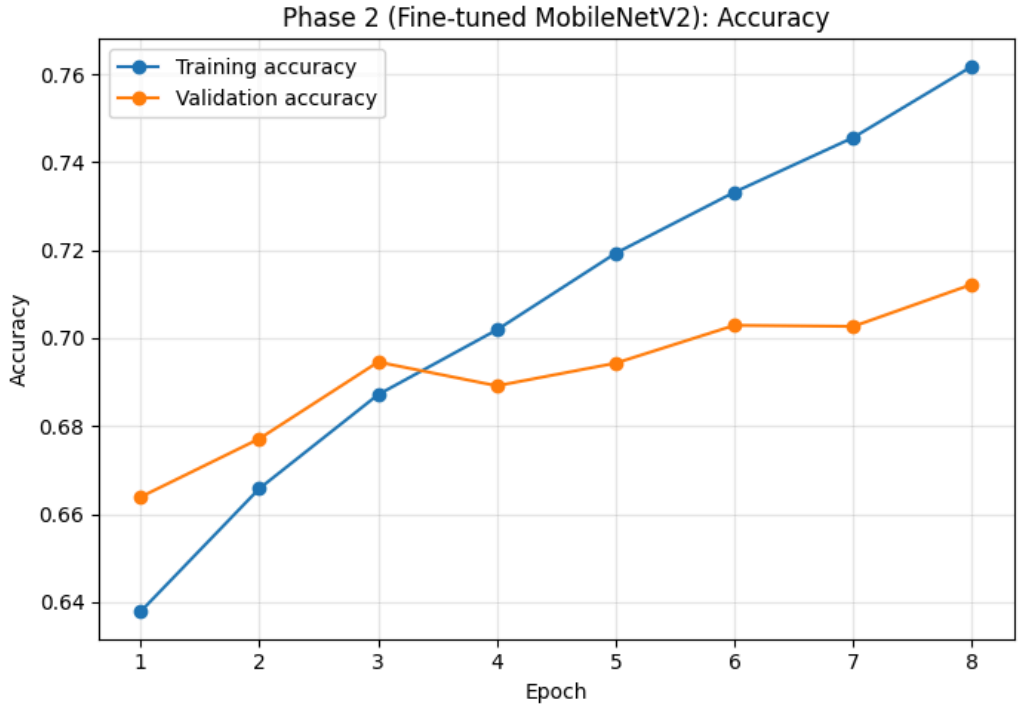
combined_train_acc = history_head.history["accuracy"] + history_fine.history["accuracy"]
combined_val_acc = history_head.history["val_accuracy"] + history_fine.history["val_accuracy"]
combined_train_loss = history_head.history["loss"] + history_fine.history["loss"]
combined_val_loss = history_head.history["val_loss"] + history_fine.history["val_loss"]

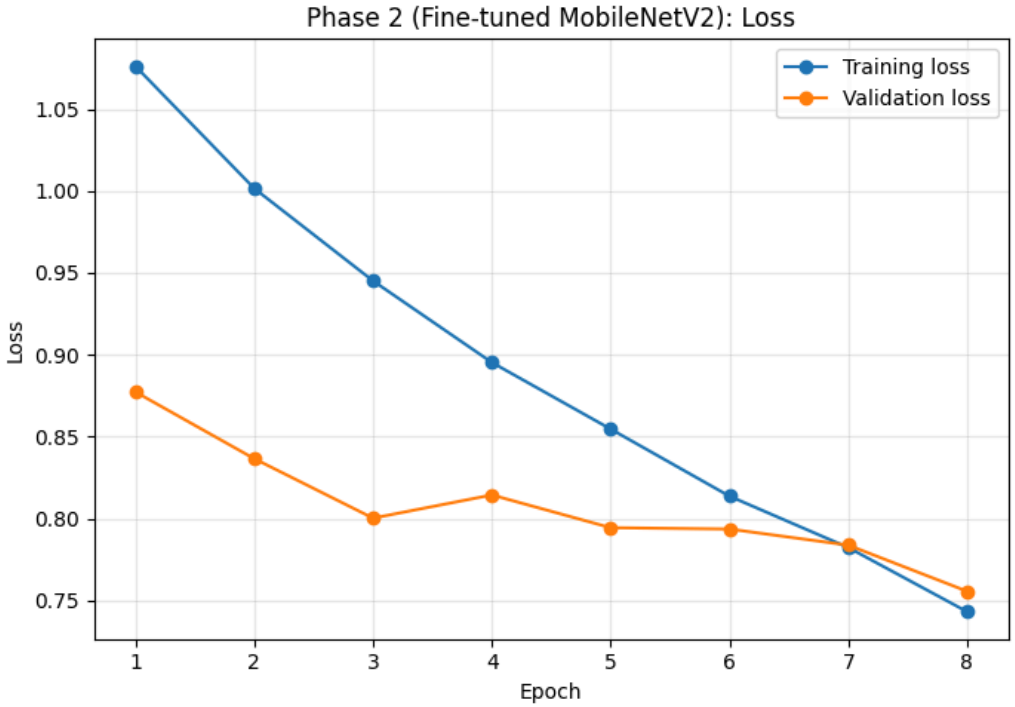
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
axes[0].plot(all_epochs, combined_train_acc, label="Training accuracy")
axes[0].plot(all_epochs, combined_val_acc, label="Validation accuracy")
axes[0].axvline(head_epochs + 0.5, color="gray", linestyle="--", label="Fine-tuning starts")
axes[0].set_title("Accuracy Across Both Training Phases")
axes[0].set_xlabel("Sequential epoch")
axes[0].set_ylabel("Accuracy")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

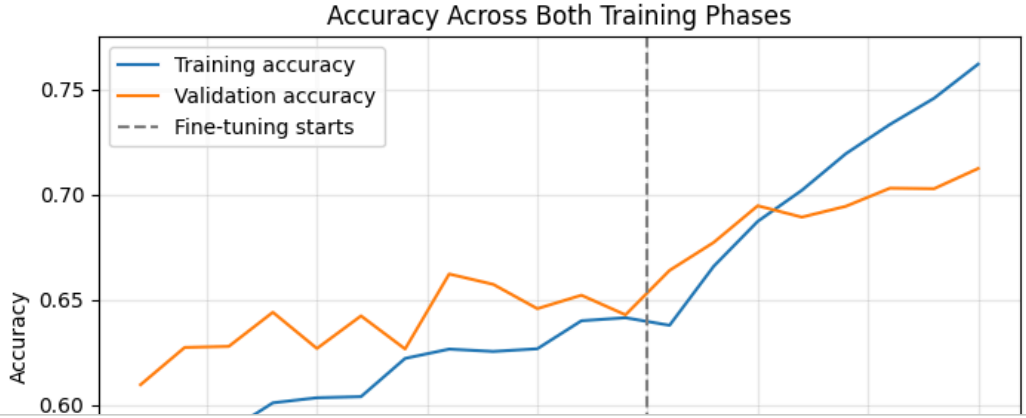
axes[1].plot(all_epochs, combined_train_loss, label="Training loss")
axes[1].plot(all_epochs, combined_val_loss, label="Validation loss")
axes[1].axvline(head_epochs + 0.5, color="gray", linestyle="--", label="Fine-tuning starts")
axes[1].set_title("Loss Across Both Training Phases")
axes[1].set_xlabel("Sequential epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
axes[1].grid(True, alpha=0.3)

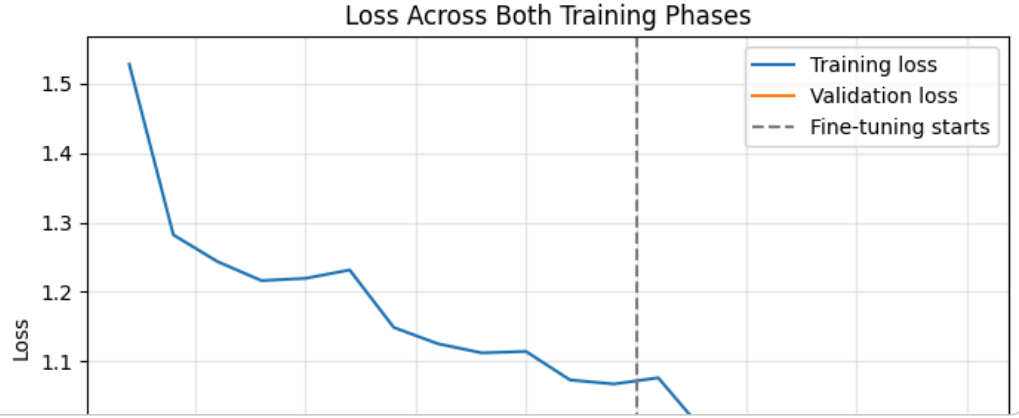
plt.tight_layout()
plt.savefig(FIGURE_DIR / "10_combined_learning_curves.png", dpi=200, bbox_inches="tight")
plt.show()
```






```
Best validation-loss epoch: 8
Best validation loss: 0.7556
Best validation accuracy: 71.23%
```





Stage 17 — Select the model using validation performance

Method: evaluate the Phase 1 and Phase 2 checkpoints on the validation set and select the one with the lower validation loss. Do not use test results to choose between models.

```
# Stage 17: Compare saved checkpoints using validation loss only.
if not HEAD_PATH.exists():
    raise FileNotFoundError(f"Missing Phase 1 checkpoint: {HEAD_PATH}")
```

```
head_model = tf.keras.models.load_model(str(HEAD_PATH))
head_val_loss, head_val_accuracy = head_model.evaluate(val_ds, verbose=0)

candidates = [{
    "name": "Phase 1 — frozen backbone",
    "path": HEAD_PATH,
    "val_loss": float(head_val_loss),
    "val_accuracy": float(head_val_accuracy)
}]

if FINE_PATH.exists():
    fine_model = tf.keras.models.load_model(str(FINE_PATH))
    fine_val_loss, fine_val_accuracy = fine_model.evaluate(val_ds, verbose=0)
    candidates.append({
        "name": "Phase 2 — fine-tuned",
        "path": FINE_PATH,
        "val_loss": float(fine_val_loss),
        "val_accuracy": float(fine_val_accuracy)
    })

comparison_df = pd.DataFrame(candidates)
display(comparison_df)

# Select the checkpoint with the lowest validation loss.
best_candidate = min(candidates, key=lambda item: item["val_loss"])
model = tf.keras.models.load_model(str(best_candidate["path"]))

print("Selected model:", best_candidate["name"])
print("Selected checkpoint:", best_candidate["path"])
print(f"Validation loss: {best_candidate['val_loss']:.4f}")
print(f"Validation accuracy: {best_candidate['val_accuracy'] * 100:.2f}%")
```

	name	path	val_loss	val_accuracy
0	Phase 1 — frozen backbone	C:\Users\Akhilesh\Datasets\best_skin_mobilenet...	0.887756	0.662165
1	Phase 2 — fine-tuned	C:\Users\Akhilesh\Datasets\best_skin_mobilenet...	0.755613	0.712251

```
Selected model: Phase 2 — fine-tuned
Selected checkpoint: C:\Users\Akhilesh\Datasets\best_skin_mobilenetv2_finetuned.keras
Validation loss: 0.7556
Validation accuracy: 71.23%
```

Stage 18 — Final test-set evaluation

Method: evaluate the selected model exactly once on the held-out test set. The test set should not be used to tune model settings.

```
# Stage 18: Report final test performance for the validation-selected model.
test_loss, test_accuracy = model.evaluate(test_ds, verbose=1)

print(f"Final test loss: {test_loss:.4f}")
print(f"Final test accuracy: {test_accuracy * 100:.2f}%")

# Generate predicted class probabilities and predicted class IDs.
```

```
y_true = np.asarray(y_test, dtype=int)
y_probabilities = model.predict(test_ds, verbose=1)
y_pred = np.argmax(y_probabilities, axis=1)

# Detailed per-class precision, recall, F1-score, and support.
report_text = classification_report(
    y_true,
    y_pred,
    labels=list(range(NUM_CLASSES)),
    target_names=class_names,
    digits=3,
    zero_division=0
)
print("\nClassification report:")
print(report_text)

# Macro metrics treat each class equally; weighted metrics account for support.
precision, recall, f1, support = precision_recall_fscore_support(
    y_true,
    y_pred,
    labels=list(range(NUM_CLASSES)),
    zero_division=0
)

print(f"Macro F1-score: {np.mean(f1):.3f}")
print(f"Overall accuracy from predictions: {accuracy_score(y_true, y_pred) * 100:.2f}%")
```

```
255/255 [==============================] - 22s 78ms/step - loss: 0.7281 - accuracy: 0.7223
Final test loss: 0.7281
Final test accuracy: 72.23%
255/255 [==============================] - 17s 65ms/step

Classification report:
                                                        precision    recall  f1-score   support

                                          1. Eczema 1677      0.542     0.409     0.466       252
          10. Warts Molluscum and other Viral Infections - 2103      0.580     0.648     0.612       315
                                       2. Melanoma 15.75k      0.796     0.913     0.851       471
                                3. Atopic Dermatitis - 1.25k      0.503     0.487     0.495       189
                           4. Basal Cell Carcinoma (BCC) 3323      0.793     0.852     0.821       499
                              5. Melanocytic Nevi (NV) - 7970      0.934     0.825     0.876      1195
                  6. Benign Keratosis-like Lesions (BKL) 2624      0.585     0.663     0.622       312
   7. Psoriasis pictures Lichen Planus and related diseases - 2k      0.482     0.510     0.495       308
         8. Seborrheic Keratoses and other Benign Tumors - 1.8k      0.708     0.675     0.691       277
9. Tinea Ringworm Candidiasis and other Fungal Infections - 1.7k      0.555     0.592     0.573       255

                                                       accuracy                          0.722      4073
                                                      macro avg      0.648     0.657     0.650      4073
                                                   weighted avg      0.729     0.722     0.723      4073

Macro F1-score: 0.650
Overall accuracy from predictions: 72.23%
```

Stage 19 — Graph the confusion matrix

Graph 10: confusion matrix. Run after Stage 18. Diagonal values are correct predictions; off-diagonal values show which classes are confused with one another.

```
# Stage 19: Plot raw and row-normalized confusion matrices.
cm = confusion_matrix(y_true, y_pred, labels=list(range(NUM_CLASSES)))

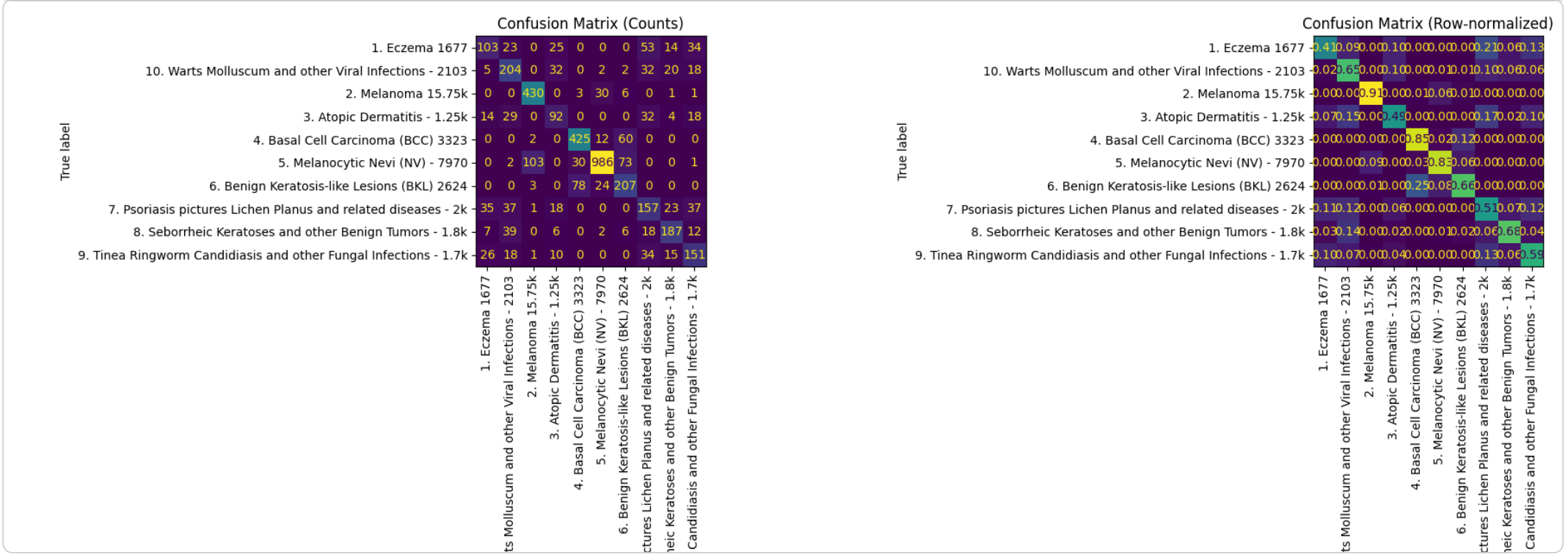
# Normalize each true-class row to show the proportion predicted in each class.
row_totals = cm.sum(axis=1, keepdims=True)
cm_normalized = np.divide(
    cm.astype(float),
    row_totals,
    out=np.zeros_like(cm, dtype=float),
    where=row_totals != 0
)

fig, axes = plt.subplots(1, 2, figsize=(20, 8))

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
).plot(
    ax=axes[0],
    xticks_rotation=90,
    values_format="d",
    colorbar=False
)
axes[0].set_title("Confusion Matrix (Counts)")

ConfusionMatrixDisplay(
    confusion_matrix=cm_normalized,
    display_labels=class_names
).plot(
    ax=axes[1],
    xticks_rotation=90,
    values_format=".2f",
    colorbar=False
)
axes[1].set_title("Confusion Matrix (Row-normalized)")

plt.tight_layout()
plt.savefig(FIGURE_DIR / "11_confusion_matrices.png", dpi=200, bbox_inches="tight")
plt.show()
```



Stage 20 — Graph per-class precision, recall, and F1-score

Graph 11: per-class metrics. This is especially useful for presentation because overall accuracy can hide classes that the model frequently misses.